



Fakultät Informatik

From CAD Software to Slicer

Workflow Optimization using Print Hints in 3MF Files for 3D Printing

Bachelorarbeit im Studiengang Medieninformatik

vorgelegt von

Luise Hartdegen

Matrikelnummer 364 2168

Erstgutachter: Prof. Dr. Ulrich von Zadow

Zweitgutachter: Prof. Dr. Timo Götzelmann

© 2025

Dieses Werk einschließlich seiner Teile ist **urheberrechtlich geschützt**. Jede Verwertung außerhalb der engen Grenzen des Urheberrechtsgesetzes ist ohne Zustimmung des Autors unzulässig und strafbar. Das gilt insbesondere für Vervielfältigungen, Übersetzungen, Mikroverfilmungen sowie die Einspeicherung und Verarbeitung in elektronischen Systemen.

Abstract

//TODO shorter motivation, more focus on realized solution; keep it general → don't mention 3MF, OrcaSlicer, Fusion

With the rising popularity and easier access to 3D printers, more and more people are interested in designing and printing their own models. There are however still a few steps involved that require domain specific knowledge, oftentimes creating a discrepancy between the wanted outcome and reality. Preparing a 3D model for printing requires the user to apply material presets and printing parameters after importing the model to a slicer software. Doing this once is a logical and necessary step, but when creating a 3D model from scratch or changing one, it might take several iterations until the result matches the desired outcome. Thus the user has to repeatedly apply the aforementioned settings and therefore they have to remember them or write them down which creates friction in the workflow. Not only does this increase cognitive workload, leading to a higher frequency of errors and thus produces waste, it also often leads to frustration in beginner or casual makers. The solution presented in this paper are so-called print hints added in the 3D modeling software itself, in the proof of concept realized using Fusion appearances. Upon opening a 3MF file in OrcaSlicer, the appearances are mapped to either material presets or modifiers. By giving the user the option to map manually while also saving and loading previous mappings, the solution provides an automation of the criticized workflow while keeping the freedom to easily change an entire group of objects in one simple step.

Kurzdarstellung

//TODO translation

Contents

1. Introduction	1
2. Background and Related Work	4
2.1. State-of-the-art Workflow	4
2.2. 3D Modeling Software	6
2.3. Slicer	6
2.4. File Formats used in 3D Printing Context	8
2.4.1. STL	8
2.4.2. OBJ	8
2.4.3. 3MF	9
2.5. Workflow using Autodesk Fusion and OrcaSlicer	9
2.5.1. <i>Autodesk Fusion</i>	9
2.5.2. OrcaSlicer	10
2.5.3. Exporting as STL	10
2.5.4. Exporting as OBJ	10
2.5.5. Exporting as 3MF	11
2.5.6. Applying Materials	12
2.5.7. Applying Settings with and without Presets	12
2.6. Related Work and Available Solutions	12
3. Concept	14
3.1. Adding Print Hints in Autodesk Fusion	14
3.2. 3MF as Preferred File Format	14
4. Technical Implementation	15
4.1. Technical Analysis of Fusion's 3MF Export	15
4.2. Code Analysis of 3MF Processing in OrcaSlicer	16
4.3. Code Changes in OrcaSlicer	16
4.4. Writing print hints to 3MF file in Fusion	17
4.5. Analysis of OrcaSlicer's 3MF File Processing	17
4.6. Implementation Design	18
5. Results	19
5.1. Outlook	19

6. Conclusion	20
A. Supplemental Information	21
List of Figures	25
List of Tables	26
List of Listings	27
Bibliography	28

Chapter 1.

Introduction

It is one of the clearest paradoxes of modern work that computer-based systems designed to reduce task complexity and cognitive workload actually often impose even greater demands and stresses on the very individuals they are supposed to be helping.[1, p. 222]

Recent years have seen a rising popularity of digital fabrication within the hobbyist or so-called maker community, as well as in educational and professional settings. This growth can be attributed to the increasing availability of devices such as 3D printers, CNC milling machines and laser cutters in public spaces, along with the declining cost of these devices for end-users.

“Like traditional printers they bridge the realms of the digital and the physical by transforming digital designs into physical artifacts. While traditional printing results in a representation of text or graphics on paper, 3D printers make use of filament to build a three-dimensional artifact layer by layer.”[Machine Sensibility]

The applications of this technology are diverse, ranging from toys to household items, from replacements components to artistic creations, from educational models to prototypes for mass manufacturing, and ultimately to finished products in the emerging market of customised mass production, exemplified by websites such as *Thingiverse.com*, where users can upload their 3D models.

Initially, when the technology became widely available, the user base consisted primarily of individuals who were not only interested in the final result, but also demonstrated a keen interest in the technology itself and the process of creating models and setting up the printer. It is from this community that the so-called *makerspaces* emerged. These are spaces that provide affordable access to digital fabrication tools and act as knowledge hubs, often with financial support from their users.

Since then, there has been a considerable enhancement in the usability and automation of 3D printers¹, thereby rendering the technology more accessible to casual users. As

¹The term “3D printers” used in this paper specifically refers to smaller desktop machines that use Fused Filament Fabrication (FFF) / Fused Deposition Modeling (FDM) technology

was the case with programming in previous decades, 3D printing has found its way into education to teach various skills, including design, engineering and project management. “An additional benefit of digital fabrication is that it accelerates the processes of ideation and invention. It eliminates manual dexterity as the ‘middleman’ in transforming an idea into a product, so students can focus their attention on improving the design rather than taking care of mundane issues with the materials – and many more cycles of redesign are possible in the same time interval.”[FabLab of Machines Maker]

Nowadays 3D printers are oftentimes available in schools and libraries as well, where the staff might not be as knowledgeable or well trained as in makerspaces. This leads to a more diversified user base in terms of knowledge, intent to learn and focus on technology versus a means to an end, i. e. the final result.

Despite these differences, several main steps can be identified in the workflow from idea to finished product, that generally consists of the same steps across the board. Using a simple box with a snap-fit closure and multiple material colors as an example, the workflow generally could look like this: Upon coming up with an idea, the user designs a 3D model using computer-aided design software. Setting it up for printing, they choose different materials for the box and the text and apply parameters for the specific materials in use in case they differ from the presets. They then use the green boxes as modifiers to increase the amount of wall loops for more stability, before having the software translate the 3D model into printer-specific instructions. After physically setting up the printer, the print is started and will result in a physical product. This product might not conform exactly to the user’s intended result, hence the product needs to be evaluated and the results thereof mirrored back into the process above, i. e. changing the model itself or refining the printing parameters.[4, p. 52] This results in an iterative process with some repetitive steps requiring the user to manually document or remember highly specific parameters.

Due to the highly specific use cases and quite different requirements, a “standard 3D printing workflow [...] requires disparate pieces of software, which then leave behind only a fragmented record of the fabrication process. CAD models are stored and shared as static geometry files. Separately, operators can tweak relevant settings in CAM software called a slicer, which can then be saved as a slicing profile.”[its not the shape]

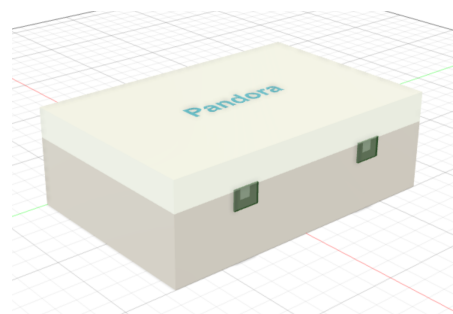


Figure 1.1.: 3D model as example

With a wider and more diverse user base, discussions about how to streamline the process, lower hurdles and simplify the sometimes rather complex steps involved have increased in the

scientific HCI² community.[2] Even though the technology itself improved in regards to usability and maintainability, the skills required to operate a 3D printer remain high. [PrintAssist]

Various different approaches have been documented to tackle this problem, ranging from easier 3D modeling software, improved process documentation[its not the shape] or even chatbots as human-machine interfaces. [PrintAssist] “[C]urrent tools that focus on supporting only one aspect of the workflow (*e. g.* , only modelling or only printing) may be less useful for casual makers and there is opportunity in HCI research to innovate on design tools that take into account the interdependencies within the 3D printing workflow”[2] //TODO check if still up-to-date / relevant

One such aspect concerns printing parameters and particularly the need for documentation either to be able to recreate the conditions or to get support with previous configurations in case something went wrong. To have consistent results when comparing different iterations of one’s print, printing parameters shouldn’t changed unless specifically identified as a problem. This does however raise the need for users to either remember their settings or document them in some way, oftentimes outside of the software used. Partly slicer software provides an option to save settings in the form of user presets but as soon as the project becomes more complex, the user still has to remember or document which presets they used for which parts. [3D Printers as Sociable Technologies]

Focusing on the connection between parts of the model and applied settings, the solution presented in this paper analyses the feasibility of the use of ‘print hints’ early on in the process to improve the interdependency of modeling and slicer software and the overall user experience. Print hints are markers, added to the model during the 3D modeling phase that the slicer can process semi-automatically. The idea is to offer the user the option to map the markers to their own material or modifier presets while simultaneously saving the resulting mapping upon importing the model to the slicer. Thus if the same maker is reused in a new iteration or even a new model, the interface can apply the mapped preset while still giving the user the option to change it.

The goal is to streamline the iterative design workflow by reducing the need to repeatedly configure the same printing parameters in each iteration, therefore lowering cognitive workload for users. The prototype presented in the following thesis focuses mainly on improving the workflow using already available options while simultaneously introducing a new user interface and consequently changed handling when opening the 3d model in the slicer.

²human-computer interaction

Chapter 2.

Background and Related Work

some clever bridge from problem to technical and related background

to understand...

research indicates...

beginning the inquiry...

2.1. State-of-the-art Workflow

Let's take a detailed look on the general workflow as described in the previous chapter, further using the 3D model 1.1 as an example.

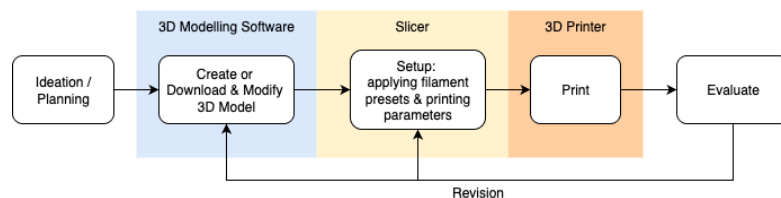


Figure 2.1.: Iterative workflow in 3D printing highlighting the step to be optimized

1. 3D Model

After coming up with an idea, the user either creates the 3D model from scratch or downloads one from the various databases available online and optionally modifies it. Creating and modifying a model requires highly domain-specific knowledge and skills. While there is a lot of educational content in the form of videos, written tutorials and documentation for every level of expertise easily available, especially for newcomers it takes a lot of practice, effort and time to reach a level where the design will come close to their desired outcome.

2. Print Setup

As the next step, they have to choose the printing material (called filament) for different parts, e. g. printing the box itself in white PLA and the text in a different color PLA, and set at the very least the temperature according to the materials specifications in case they differ from the presets used. Oftentimes further parameters or presets are needed to determine factors like supports¹, infill, speed and heating bed temperature. In this case, the green boxes around the closures need to be marked as modifiers to increase the wall loops in the main model for added stability. Furthermore the box is designed to be printed as two separate bodies, one for the top and the bottom each, which means the top including all its text bodies have to be placed on the printing bed accordingly.

3. Slicing

The model then has to be translated into layer-by-layer instructions for the printer.

4. Printing

After inserting the chosen materials into the printer and optionally calibrating, the model can be printed.

5. Evaluation

Especially for newcomers and casual makers, the print might not turn out perfect on the first try. Therefore the print has to be evaluated for design flaws or printing errors. This often leads to revisions in the 3D model itself or the print setup, starting a new iteration of the workflow.

Ergo, two parts of the process require a lot of highly specific knowledge: 3d modeling and setting the model up to print. In particular in regards to the iterative nature of the process, a few repetitive steps can be identified considering the following aspects.

1. Using multiple materials (e.g. printing the text in a different color): In each iteration the user has to apply the material presets to the various printed parts, which can become tedious and lead to mistakes when the project becomes more complex.
2. Using model parts as modifiers (e.g. the translucent green boxes as modifiers to print more wall loops in the closure's area for higher stability): Equivalent to the aspect above, the modifiers have to be set in each iteration. Unfortunately presets can't always be used for this step, resulting in even more need to document and apply settings.

¹//TODO definition

2.2. 3D Modeling Software

//TODO sources & content

Realizing an idea from scratch, rather than downloading an available model online and modifying it, the user would create their model using 3D modeling software. This software is used to transform an idea or a 2d sketch into a 3D solid or surface representation, meaning that it describes the external geometry fully. [additive manufacturing technology]

An important distinction to make when considering 3D modeling software is the target medium. Software targeting digital end products generally handle 3D models mainly based on surfaces, while software made for creating manufacturing models also take into consideration the volumes and the closure of the model's surface. Even though the latter often provides functionalities for processes such as computer-aided manufacturing and simulating physical properties as well, the computer-aided design (CAD) aspect is the only one relevant to the workflow at hand.

As such, modeling software for manufacturing purposes bases its models on precise measurements, often starting from 2D sketches with rigid constraints regarding measuring units, angles and positioning in relation to other model parts. These sketches are then extruded, cut or otherwise manipulated, thus creating three dimensional volumes.

Depending on the specific software, the final result is described either by a consecutive order of instructions or by a polygonal structure. //TODO fact check & add more info about representation

2.3. Slicer

Steps 2 and 3 of the workflow described above, are done with so-called slicer software. The name originates from its main task of slicing a 3D model into printable layers, since “the key to how [additive manufacturing]² works is that parts are made by adding material in layers; each layer is a thin cross-section of the part derived from the original CAD data.”[Additive Manufacturing Technologies] While it does offer some basic functionalities to create 3D models as well, its focus lies in setting up a finished model for printing and translating the data represented as described above into instructions for a specific target printer to follow. [2]

Slicers are capable of importing 3D models in various different representations and transforming them to the slicer's preferred format. Visually the model's parts are placed on

²//TODO explain term 'AM'

a virtual representation of the printer's heating bed, which is the surface that a model is printed upon. Since printers have different dimension, the heating bed as well as the available height need to be available to the slicer for proper setup. Users have the option of setting this up manually, however more commonly the slicer accesses a list of printers, oftentimes from various manufacturers, for the user to select one from. These dimensions are important to arrange parts in an efficient way to print, in case the model consists of multiple parts.

The software also provides a selection of generic as well as manufacturer specific filament presets containing specifications such as the filament's name and material, recommended printing nozzle and heating bed temperature, and printing speed. From these the user can create their own filament presets, tweaking parameters to accommodate for their own needs to reuse the settings across projects. Further printing parameters that need to be frequently changed include quality related parameters (layer height, line width, precision, bridging), parameters regarding the model's strength (wall loops, top and bottom shell thickness, infill amount and type) as well as the type of support and thresholds where supports need to be applied.

When the model is sufficiently set up, the slicer translates it into layers, described as paths for the printer to follow. Due to the nature of this process, the slicer handles projects in two different ways. Before slicing, the 3D model needs to be saved as a project including all parameters as defined by the user in a way, so it can be loaded and updated at a later point, independent of the printer to be used. For this step, slicers generally use either the open-source format 3MF or a proprietary file format. The sliced model file, on the other hand, needs to be specific to one printer ecosystem and is usually saved as gcode or bgcode files.

There are many different slicers available online nowadays, either general ones or specific to one printer manufacturer. Since getting all the settings right for a particular 3D printer can be difficult, some manufacturers prefer to modify one of the open source programs, creating a version for their machines.^[4, p. 58] Thus, most of the commonly used slicers today are based on the same code base, derived from an ancestor called Slic3r. Due to the similarity in code base and the need to keep these projects open-source³, useful features are oftentimes introduced in one version and then ported to the other ones.

³Slic3r is licensed under the GNU Affero General Public License, version 3, which states that every software using parts of it has to be subject to the same license.

2.4. File Formats used in 3D Printing Context

There are various different file formats available to represent 3D models in general, however not all are equally appropriate for digital fabrication. Hence, the following paragraphs will delve into the three formats used most in 3D printing.

2.4.1. STL

The STL (“STereoLithography”) format has been introduced in the 1980’s and since then acquired support by most softwares in use for digital fabrication. It is considered to be the most well-known and used mesh file format, even though it has rigorous limitations. The contents can be either encoded in binary or in ASCII text format, the latter of which is less common, but used for illustration and educational purposes. It describe an approximation of the surface geometry in the form of a mesh of triangles, each identified by a unit normal vector as well as three vertices, as seen in [A.1](#). The normal vector is equal to the length “1”, however it does not specify whether the dimensions are millimeter, inches or another unit entirely. Thus, while the proportions of a model will remain preserved, the overall size depends on the user’s knowledge about the model. Due to its simplicity, the file does not contain information regarding color, text or other common CAD model attributes.[Additive Manufacturing Technologies, p. 341 ff][3D Printing with Fusion, Chapter 5]

Furthermore, since the specification requires each triangle to share two of its vertices with the triangle adjacent to it, thus creating one continuous solid model, technically there will always be exactly one body in the resulting file. For a model comprised of multiple parts, the user therefore has to choose whether to combine all bodies into one or export each body as a separate file. However, modern slicer software is capable of separating parts into disparate bodies, even if the user chooses to export a multibody model into one file.

2.4.2. OBJ

The OBJ file format is considered to be the second most common file format as it contains color and texture information in addition to the options STL presents. As can be seen in [A.2](#), the model is represented by vertices as well. Each group of vertices is prefaced by a body name and optionally a material, referencing a separate `.mtl` file.

`//TODO //TODO //TODO`

2.4.3. 3MF

3MF has been developed by the 3MF Consortium, founded by Microsoft, Autodesk, HP, 3D Systems, Prusa Research and several other key players in the field, to create a 3D printing file format that addresses the shortcomings of previous standards (STL, OBJ, AMF). In particular the possibility for extensions and the support of colors as well as several different 3D model representations were a main factor, making it ideal for additive manufacturing.[7]

Unlike the other two formats, 3MF not only preserves separate bodies even when exported as a single file making it more user-friendly, it also includes information about the model's color, material, and textures.

“A 3MF file is a ZIP archive that contains a complex file structure, defining a 3D model in Extensible Markup Language (XML) file”[7] usually called. Optionally, there can be multiple 3D model files, one of which needs to be identified in the StartPart relationship as the root model.

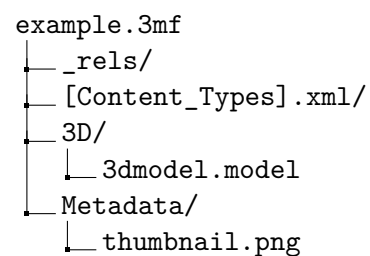


Figure 2.2.: Example structure of 3MF file

2.5. Workflow using Autodesk Fusion and OrcaSlicer

the following analysis will focus on the steps from exporting a model to printing using two specific softwares as example, highlighting in particular how the workflow differs depending on the file format used in regards to three points.

2.5.1. Autodesk Fusion

The first software, Autodesk Fusion, is used for designing, engineering, simulating electronics and as basis for manufacturing[3]

It provides functionalities for the entire product development process with support for various different techniques[3], meaning it can be used for designing parts to manufacture in either additive or subtractive processes

It also provides visualization tools for examining and analyzing designs, as well as simulation capabilities, for example, to assess how different components interact or respond to various mechanical stresses.

2.5.2. OrcaSlicer

OrcaSlicer is one the open source slicers, originally forked from *Bambu Studio*, which itself was forked from *PrusaSlicer*, a descendant of *Slic3r*. “Slic3r is licensed under the GNU Affero General Public License, version 3”[5], which means that every software using parts of it has to be licensed under the same license. Since these slicers are all intricately linked, oftentimes new features are being introduced in one of them and then pulled over into the others as well.

Exporting the model from Fusion is possible via two dialogs: the general export and the 3D print specific one. The two dialogs each offer a different set of file formats due to their different purposes. Since this analysis deals with 3D printing in particular, only the export dialog specific to 3D printing will be considered, which offers to export as STL, 3MF and OBJ format. For all three the user can decide whether to generate one file containing all selected components and bodies, or one file per body.

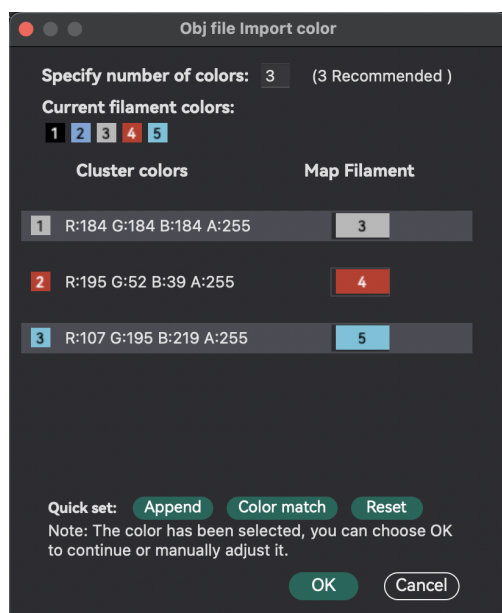
2.5.3. Exporting as STL

when combining all bodies into one STL file → one extra step to split the model into separate objects within OrcaSlicer to be able to print the text in a different color or to use the green boxes as modifiers

when importing multiple files at once → OrcaSlicer has the user confirm whether the files should be loaded as a single object with multiple parts

The advantage over loading each file as an object is the preservation of the spatial relation and thus parts, like the modifiers in the example box, can be placed at any height while a single object is always placed with at least one surface flush with the printing plate.

2.5.4. Exporting as OBJ



cite software? & check formatting for dialog

Consequently, OrcaSlicer shows a particular dialog (fig. 2.5.4) when opening an OBJ file, asking the user how they want to map the file’s colors to current or new filament presets. This makes it easy for users to align filament colors (i.e. the color the filament is represented as in

OrcaSlicer, not necessarily the color of the filament itself) to colors applied in Fusion. one body per file → in the same dialogs and extra steps.

2.5.5. Exporting as 3MF

Unlike the other two formats, 3MF not only preserves separate bodies even when exported as a single file making it more user-friendly, it also includes “information about the model’s color, material, and textures.”[6] Initially when importing a 3mf file containing multiple objects, OrcaSlicer asks the user whether the file should be loaded as a single object with multiple parts instead of considering them as multiple objects similar to the dialog when importing multiple STL or OBJ files at once.

While Fusion does export materials as RGBA colors in one hexadecimal color code per body, OrcaSlicer does not offer the user to map these to available or new presets so far. Consequently users have to reapply materials manually which has been criticized by users, e.g. in *OrcaSlicer issue #2438*⁴.

OrcaSlicer always handles files as projects, either by interpreting 3MF files as a project in itself or by opening a file within a new project. That way the user has the option to save project specific settings or presets for further use or just export the generated g-code for printing and discard said settings.

fuse next two subsections → clear difference between *filament*, *settings*, *parameters* etc if using em

⁴Single Object Multiple parts with Multiple Colours: [...] I have made a number of 3D models with multiple parts in single 3mf file. I mostly use Fusion 360 for models and during modeling I assign colours to each part. But when I upload this 3mf file to the slicer, It assigns the primary filament colour to all the parts. I was wondering if the slicer would be able to distinguish the parts with different colours and assign either the original colour to the parts or even random colours(which can be changed later). For example if my 3mf file has 5 colour parts, the slicer should be to detect them. Then it should find which colour has the maximum spread by area and assign the filament 1 to it. The second largest should be assigned the second filament and so on till the last.[...] <https://github.com/SoftFever/OrcaSlicer/discussions/2438>

2.5.6. Applying Materials

“For extrusion-based 3D printing, material properties such as rheology⁵ or melting temperature are as relevant as machine configurations such as print speed, nozzle geometry, and heating profiles.” [It’s Not the Shape, It’s the Settings]

Generally, materials can be set using system or user defined presets per object. Material settings entail, amongst others, the filament itself, nozzle temperature, general printing speed and cooling settings. OrcaSlicer actively asks the user, whether they want to save any changes to material parameters they have made, which makes it easy to save either parameters specific to one’s printer and filament or even specific to one project while simultaneously keeping default material presets untouched. When importing a new file or project, OrcaSlicer does apply the first available user preset filament by default. More detailed analysis even showed, that a multicolor file will be imported with different filaments assigned consecutively but the user doesn’t have any options to map colors to filaments the way it works with OBJ files. Furthermore, the last color per colorgroup (more details in the following chapter ??) is assigned to each object which is why even complex models with different colors appear as having one single material in OrcaSlicer.

2.5.7. Applying Settings with and without Presets

Settings like the amount of wall loops, infill percentage and pattern, and printing speed for single layers might need to be customized. It is possible to save these as presets as well, however only globally, i. e. they can only be applied to the entire project, not to specific objects within. As criticized in the *OrcaSlicer Issue #7769*⁶, particularly for projects containing multiple objects, using presets in this case is not a valid option.

2.6. Related Work and Available Solutions

“CAM-based design tools have emerged where toolpaths are specified directly. This approach is contrasted with CAD-based tools which are based on geometric models [23]. CAM-based tools naturally shift focus to the machine and its settings. Prior work generates custom toolpaths using traditional CAD environments [20 , 21], code [25 ,

⁵//TODO explain

⁶Copy/paste per-object settings: When planning/building large projects in-slicer it often becomes an extremely tedious task to click all the objects you want a specific set of settings to apply to. The larger the assembly/project you are planning, the easier it is to miss a body and make a mistake. If you "assemble" models together you cannot edit multiple sub-objects across assemblies.
<https://github.com/SoftFever/OrcaSlicer/issues/7769>

54 , 62], hand-drawn sketches [23], and domain-specific abstractions [5].”[its not the shape]

how do others solve the issue → focus on documentation [its not the shape]

→ maybe also refer to git issues here

Chapter 3.

Concept

general explanation of how I solved this

3.1. Adding Print Hints in Autodesk Fusion

using Fusion's native appearance as a proof of concept to keep it simple

pro: users can create their own library of appearances to use in various projects which makes it easy to have consistent mapping of Fusion appearance to OrcaSlicer filament preset

3.2. 3MF as Preferred File Format

it provides a lot of options by default but is also open to expansion → get some content from chap 2

plus: OrcaSlicer saves all projects as 3mf by default

what changes in the 3mf file → nothing, but in a future implementation it might be possible to add more print hints / hints not depending on specific cad software

Chapter 4.

Technical Implementation

4.1. Technical Analysis of Fusion’s 3MF Export

3MF’s 3D model per specification is comprised of two sections: “a set of resource definitions that include objects and materials, as well as a set of specific items to actually build”.[8]

Taking a closer look at the composition of the XML file as it is exported from Fusion, see A.3, these sections can be found in lines 4 ff. (the `resources` tag), enclosing the `m:colorgroup` and `object` tags, and lines 41 ff. containing the `build` tag. While materials can be represented in a number of different resource tags (e.g. `basematerials`, `m:texture2d`, `m:compositematerials...`), Fusion seems to solely use the `m:colorgroup` tag. Unfortunately there aren’t a lot of information available as to the exact way Fusion exports 3MF, hence the following paragraph is partly based on explorative observations.

The `colorgroup` contains two or more colors as an RGBA hexadecimal color code each, as mentioned before. Per the additional Materials and Properties Extension specification[9], colors are specifically expressed in sRGB colorspace in accordance with the World Wide Web Consortium. Each group is identifiable by its `id` which is referenced in each object as `pid` while the colors themselves implicitly get a 0-based index. Despite the colors’ representation as an RGBA hexadecimal color code, thus including an 8-bit channel for opacity, the standard specifies that color properties *should not* reference translucent display properties. If required, a multi-properties group must be used instead. Empirical observations indicate compliance with this requirement, as the opacity value is consistently set to FF, i. e. a fully opaque color.

Since it is possible in Fusion to apply appearances and physical materials independently of one another, one might think one color represents the appearance and the other the physical material. Observations, however, do not support this theory since even when changing both, only the first color within one `colorgroup` changes while the second one stays a light grey (“#A0A0A0FF”).

The following `object` tags each contain attributes (body id, type, color id) as well as one triangular geometry mesh representation of an object volume. The latter is represented by vertices (x-, y-, z-coordinates in the unit specified within the model's attributes) and triangles referencing three vertices each oriented with normals pointing towards the exterior of their object.^[8] Meshes of different objects may intersect (as seen in the above mentioned example's case of wall loop modifiers) or self-intersect, in which case it must be specified which areas are inside or respectively outside the volume.

In regards to adding print hints directly to the model's xml structure, it shall be noted that in accordance with the specification "XML content MUST NOT contain elements or attributes drawn from the "xml" or "xsi" namespaces unless they are explicitly defined in the XSD schema or by other means in the specification."^[8]

4.2. Code Analysis of 3MF Processing in OrcaSlicer

how does orca slicer handle 3mf files on a code level

→ read in xml lines and save them to model

→ write colors to extruders consecutively using user presets (saved as json and smth else files) → why didn't it do that before? it used a standard grey color instead of the first user preset, didn't it?

4.3. Code Changes in OrcaSlicer

first step: fix reading in the model's colors correctly → after in depth analysis I noticed that it does read in the color, but always used the last color per colorgroup tag ⇒ since Fusion adds a grey material to all colorgroups this caused OrcaSlicer to overwrite the user's color choice

next step: adding a new user interface inspired by the color mapping dialog for OBJ files with the addition of a dropdown selection field for modifiers instead of filaments

also: saving the mapping as applied in said interface to a file similar to user filament presets

The task of setting printing parameters is by nature a complex one, due to the many different printers, materials and project-specific needs out there. To recap briefly, the intention is not to reduce that complexity in and of itself, but to reduce the need to remember and repeatedly apply those settings. Accordingly from the very start, the

idea was to mark bodies with so-called ‘print hints’ in a 3D model in a way that enables the slicer to apply the user’s parameters automatically when importing a new model.

option 1: writing directly to the 3mf `model_settings.config` → complex

option 2: using Fusion’s native options to apply appearances and mapping those to filaments, printing parameters and modifier types using a custom mapping file and a dialog similar to the filament mapping when importing an .obj file

4.4. Writing print hints to 3MF file in Fusion

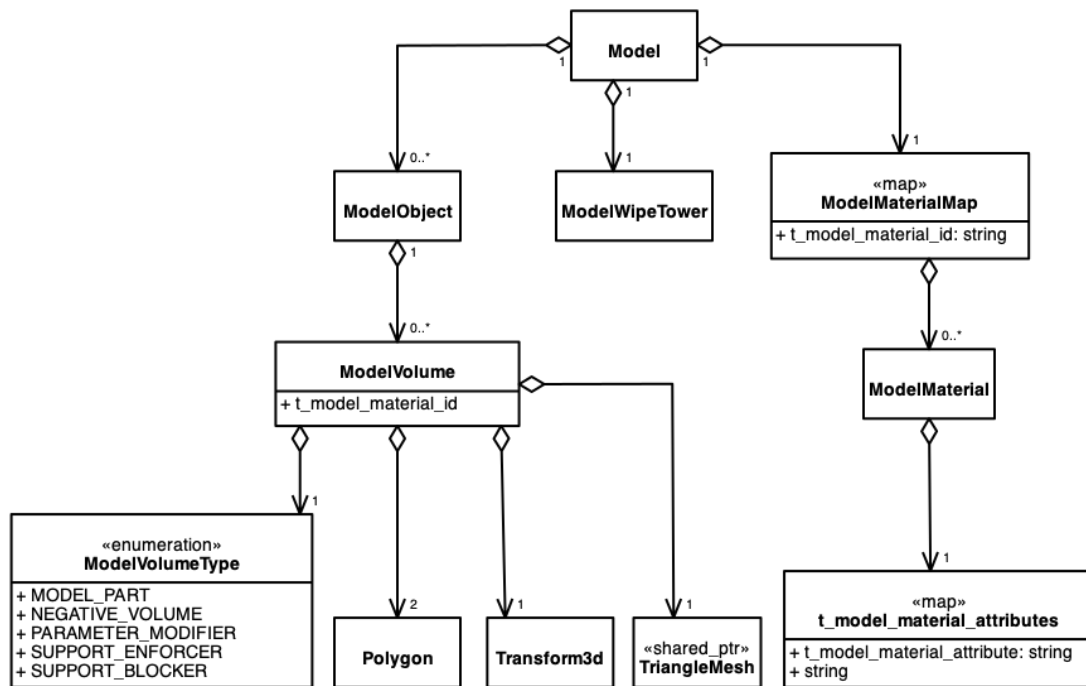
as a proof of concept → using Fusion’s native appearances (as opposed to physical materials) to mark differentiate bodies with diverging settings

these are already being saved and the big advantage is that users can create a personal library of appearances for all there models to reuse the appearance-to-parameter mapping

4.5. Analysis of OrcaSlicer’s 3MF File Processing

While chapter ?? put a focus on the state-of-the-art workflow from the user’s perspective, coming up with a way to solve the problem required a deeper analysis of the way OrcaSlicer handles 3mf files since this provided some clues as well as limitations regarding possible approaches.

- it does read in all the xml lines from the *3dmodel.model* file including the materials but it doesn’t write them to the model
- the colors are being read in but due to the way Fusion saves colors in 3mf files (as explained previously), OrcaSlicer iterates over all colors per colorgroup and overwrites the ‘correct’ (first) color with the default grey which is usually the last color per colorgroup
- this was the easier part and first step to fix
- it does however map the colors automatically to extruders and assigns the user’s saved filaments consecutively



4.6. Implementation Design

different possible approaches to solve the problem at hand and why did I do it the chosen way?

As mentioned before, as a proof of concept using specific materials to represent the possible modifiers was the way to go. The idea was to save user presets locally containing a map of “materials”, i.e. the RGBA hexadecimal color codes, to chosen modifier presets or actual filaments respectively. When opening a new 3MF project, the user would be prompted to either confirm the material to modifier / filament mappings or select options for newly detected materials. This reduces the need to apply settings upon each import while simultaneously giving the user the option to easily change mappings.

- software architecture (diagram?)
- input, output
- choices that had to be made

Chapter 5.

Results

- what does the optimized workflow look like? → example
- why is the solution good? → expert interviews
 - how are the results analysed / validated?
 - openly available → advantages of that
 - but: it is a proof of concept / prototype and thus will most likely not be the end of the story

5.1. Outlook

- communication with OrcaSlicer's owner softfever?
- option of code being pulled into other slicers
- what is left to do?

Chapter 6.

Conclusion

introduction backwards with a focus on results

Appendix A.

Supplemental Information

```
1 solid ASCII
2   facet normal 1.000000e+00 0.000000e+00 -0.000000e+00
3     outer loop
4       vertex 9.900000e+01 -3.480000e+01 3.250000e+01
5       vertex 9.900000e+01 -3.480000e+01 3.500000e+01
6       vertex 9.900000e+01 -4.020000e+01 3.250000e+01
7     endloop
8   endfacet
9   facet normal 1.000000e+00 0.000000e+00 0.000000e+00
10    outer loop
11      vertex 9.900000e+01 -4.020000e+01 3.250000e+01
12      vertex 9.900000e+01 -3.480000e+01 3.500000e+01
13      vertex 9.900000e+01 -4.020000e+01 3.500000e+01
14    endloop
15  endfacet
16  ...
17 endsolid
```

Listing A.1: Abbreviated Example of STL's 3D Model Representation

```
1 # WaveFront *.obj file (generated by Autodesk ATF)
2
3 mtl lib Pandoras_Box.mtl
4
5 g Body1
6
7 v 99.000000 -34.800000 32.500000
8 v 99.000000 -34.800000 35.000000
9 ...
10 vt 0.000000 0.000000 0.000000
11 vt 0.250000 0.000000 0.000000
```

```

12 ...
13 vn 0.100000 0.000000 0.000000
14 vn 0.100000 0.000000 0.000000
15
16 usemtl Nylon_12_(with_Formlabs_Fuse_1_3D_Printer)
17 f 1/1/1 2/2/2 4/3/3
18 f 4/3/3 2/2/2 3/4/4
19 ...
20
21 g Body2
22
23 v 99.000000 -35.000000 35.000000
24 v 99.000000 -35.000000 32.500000
25 ...
26 vt -0.250000 0.000000 0.000000
27 vt -0.000000 0.000000 0.000000
28 ...
29 vn -0.100000 0.000000 0.000000
30 vn -0.100000 0.000000 0.000000
31 ...
32 usemtl Nylon_12_(with_Formlabs_Fuse_1_3D_Printer)_(1)
33 f 369/833/809 370/834/810 372/835/811
34 f 372/835/811 370/834/810 371/836/812
35 ...
36
37 # 3464 vertices
38 # 7296 texture params
39 # 7036 normals
40 # 6904 facets
41
42 # 11 groups
43
44 # WaveFront *.mtl file (generated by Autodesk ATF)
45
46 newmtl Nylon_12_(with_Formlabs_Fuse_1_3D_Printer)
47 Kd 0.705882 0.701961 0.701961
48
49 newmtl Nylon_12_(with_Formlabs_Fuse_1_3D_Printer)_(1)
50 Kd 0.941176 0.968627 0.972549

```

Listing A.2: Abbreviated Example of Wavefront OBJ's Representation of a 3D Model and Material Collection

```

1 <?xml version="1.0" encoding="utf-8"?>
2 <model xmlns="http://schemas.microsoft.com/3dmanufacturing/core
  /2015/02" unit="millimeter" xml:lang="en-US" ...>
3   <metadata name="Title" type="QName" preserve="1">Pandoras
     Box</metadata>
4   <resources>
5     <m:colorgroup id="2">
6       <m:color color="#B4B3B3FF" />
7       <m:color color="#A0A0A0FF" />
8     </m:colorgroup>
9     <m:colorgroup id="4">
10      <m:color color="#449648FF" />
11      <m:color color="#A0A0A0FF" />
12    </m:colorgroup>
13    ...
14    <object id="1" name="Body1" type="model" p:UUID="88b92134
      -b51b-4129-a501-3cb7a6b35a19" pid="2" pindex="0">
15      <mesh>
16        <vertices>
17          <vertex x="99.000000" y="-34.800000" z="
            32.500000" />
18          <vertex x="99.000000" y="-34.800000" z="
            35.000000" />
19          ...
20        </vertices>
21        <triangles>
22          <triangle v1="0" v2="1" v3="3" />
23          <triangle v1="3" v2="1" v3="2" />
24          ...
25        </triangles>
26      </mesh>
27    </object>
28    <object id="3" name="Body10" type="model" p:UUID="83
      b1ffc1-fe3a-41ff-b2d8-709e401b58c0" pid="4" pindex="0"
29    >
      <mesh>

```

```

30         <vertices>
31             <vertex x="97.000000" y="44.000000" z="26.000000"
32                 " />
33             ...
34         </vertices>
35         <triangles>
36             <triangle v1="0" v2="1" v3="3" />
37             ...
38         </triangles>
39     </mesh>
40 </object>
41 </resources>
42 <build p:UUID="9062348e-a030-4f92-86b2-786a03cd804d">
43     <item objectid="1" p:UUID="df0ef4a8-b05c-41c3-a041-
44         f6ca9d41a68c" />
45     <item objectid="3" p:UUID="2dd63bd5-069d-405e-9b05-0
46         fd5f5a71f72" />
47     ...
48 </build>
49 </model>

```

Listing A.3: Abbreviated Example of 3MF's 3dmodel.model File

List of Figures

1.1. 3D model as example	2
2.1. Iterative workflow in 3D printing highlighting the step to be optimized	4
2.2. Example structure of 3MF file	9
2.3. OBJ color mapping dialog in OrcaSlicer	10

List of Tables

List of Listings

A.1. Abbreviated Example of STL's 3D Model Representation	21
A.2. Abbreviated Example of Wavefront OBJ's Representation of a 3D Model and Material Collection	21
A.3. Abbreviated Example of 3MF's 3dmodel.model File	23

Bibliography

- [1] J. A. Jacko, *Human Computer Interaction Handbook: Fundamentals, Evolving Technologies, and Emerging Applications, Third Edition*. Milton, UNITED KINGDOM: Taylor & Francis Group, 2012. [Online]. Available: <http://ebookcentral.proquest.com/lib/thnuernberg/detail.action?docID=911990>
- [2] N. Hudson, C. Alcock, and P. K. Chilana, “Understanding Newcomers to 3D Printing: Motivations, Workflows, and Barriers of Casual Makers,” in *Proceedings of the 2016 CHI Conference on Human Factors in Computing Systems*, ser. CHI ’16. New York, NY, USA: Association for Computing Machinery, 2016, pp. 384–396. [Online]. Available: <https://dl.acm.org/doi/10.1145/2858036.2858266>
- [3] Autodesk Inc., “Autodesk Fusion | 3D CAD, CAM, CAE, & PCB Cloud-Based Software | Autodesk.” [Online]. Available: <https://www.autodesk.com/products/fusion-360/overview>
- [4] J. Horvath and R. Cameron, *Mastering 3D Printing in the classroom, library, and lab*. New York: Apress, 2018.
- [5] SoftFever, RF47, ianalexis, and Arcodude, “OrcaSlicer/doc.” [Online]. Available: <https://github.com/SoftFever/OrcaSlicer/tree/main/doc>
- [6] C. Iancu, “ABOUT 3D PRINTING FILE FORMATS,” *Annals of the „Constantin Brancusi” University of Targu Jiu*, no. 2, pp. 135–138, 2018.
- [7] J. Rossel, V. Mladenov, and J. Somorovsky, “Security Analysis of the 3MF Data Format,” in *Proceedings of the 26th International Symposium on Research in Attacks, Intrusions and Defenses*. Hong Kong China: ACM, Oct. 2023, pp. 179–194. [Online]. Available: <https://dl.acm.org/doi/10.1145/3607199.3607216>
- [8] 3MF Consortium, “3MF Core Specification & Reference Guide,” International Organization for Standardization, Standard, Feb. 2025. [Online]. Available: <https://3mf.io/spec/core-v1-3-0/>
- [9] —, “Materials Extension,” International Organization for Standardization, Standard, Feb. 2025. [Online]. Available: <https://3mf.io/spec/materials-v1-2-1/>

- [10] T. Kosch, J. Karolus, J. Zagermann, H. Reiterer, A. Schmidt, and P. W. Woźniak, “A Survey on Measuring Cognitive Workload in Human-Computer Interaction,” *ACM Computing Surveys*, vol. 55, no. 13s, pp. 1–39, Dec. 2023. [Online]. Available: <https://dl.acm.org/doi/10.1145/3582272>